

Advanced Machine Learning for Physics student project: RECENTRE

Francesco Passante and Eugenio Nicotina
(Dated: June 23, 2026)

We build a machine learning framework to train, validate, finetune models and perform data analysis for the RECENTRE (REal-time motion CorrEction in magNeTic REsonance) project. We first reproduce the original results of the RECENTRE project based on a GRU network through a config-oriented software design, then we introduce and evaluate more modern machine learning architectures, studying in what regimes they are most effective and whether they're better candidates for real time deployment in MRI machines. Model size, inference latency, robustness to noise and uncertainty calibration are among the many different metrics by which we evaluate the performances of each of the proposed architectures.

I. INTRODUCTION

The RECENTRE project is concerned with real time prediction and correction of head motion parameters. A crucial fact to take into account when building models for this task is the inference latency of the model. The temporal distance between the frame recording and the subsequent next frame prediction must be of the order of 100 ms, so the inference latency must be well below this limit to account for other software- and hardware-related latencies.

The main goals of the present project are to reproduce the original paper's ([1]) results, which are based on a GRU network, and evaluate modern machine learning architectures such as transformers, TCNs, and physics-informed models to study whether they are better candidates for the project's task.

Given the large amount of models and the large amount of training runs (different model hyperparameters, different train / test tasks, data augmentation, etc...) we introduced a config-based software design pattern to keep the code and the output data well organized. This way, new models can be implemented and registered in a model registry dictionary; the hyperparameters of the model, the preprocessing pipeline of the data and the training details (epochs, learning rate, ...) can all be specified in a minimal, easy-to-read configuration file. This way, we built a solid end-to-end pipeline from the original raw data and model specification to the full training and evaluation loop.

II. DATASET

The raw dataset is taken from the Human Connectome Project (HCP), which consists in navigator data taken by MRI machines with a sampling frequency of 720 ms. Each 'frame' has 6 motion parameters: 3 of them are translation parameters describing the head position of the patient, the remaining 3 are rotation parameters indicating the inclination of the patient's head. The dataset contains three different scenarios: the "R" (resting) one, where patients were resting while the MRI scan was running, then the "M" (memory) one, where patients

were recollecting memories, and the "L" (language) one, where patients were actively talking. There are a total of 3221 time series among the three tasks. Each time series is associated with a patient. The preprocessing pipeline discards 140 time series, so that 1027 patients are all associated with the 3 time series of the "R", "M", "L" tasks. The three tasks have different time lengths. The R data is 1200 frames long, while the M and L are 405 and 316 frames long, respectively. Lots of efforts were put in the thesis and paper ([2]) to study different data splits for training and testing, e.g. considering the "cross-task" scenario or "cross-patient" scenario. We build on their findings (no substantial difference in performance among the different scenarios) by:

1. Developing a minimal, easy pipeline to choose training and testing tasks and choosing whether to use different splits of patients
2. Standardizing all results to the same scenario: train on all tasks (R+M+L) and test on all tasks (R+M+L), while keeping disjoint sets of patients to avoid any possible leakage

A subsequent analysis on physics-inspired models has shown that exploiting additional kinematic data such as velocity and acceleration parameters substantially improved test performances. So, 6 additional columns (already present in the raw dataset) containing velocity parameters for the 6 different dimensions are kept and used (the previous work discarded this data).

A. Data augmentation

The dataset contains roughly 2 million input window - frame target pairs, so data augmentation is not a priority for the project. Nevertheless, we investigated whether data augmentation could provide a benefit to model performances. Since the translation and rotation parameters are physically linked, there is no much room for independent transformations of the input data. However, two possible transformations are considered for data augmentation: time reversal and sign flip. These two augmentation techniques can quadruple the initial training dataset.

B. Preprocessing

The preprocessing pipeline consists in extracting the raw data from the HCP files, converting degrees to radians, ensuring a standardized time length for each of the different tasks, and ensuring that each patient is associated with its three runs. The preprocessing pipeline is meant to be run only once, as it builds three processed dictionaries (one for each task) that associate each pa-

tient id with its time series for that task.

III. METHODOLOGY

IV. RESULTS

V. DISCUSSION

VI. CONCLUSIONS